

BBP-SHA VM: Backwards Render Collapse

Full executable notebook

This notebook is a **full executable research notebook** for the manuscript “**The Ontological Inversion: Digital Physics, Reversible Cryptography, and the Universe as a Pre-Computed Mathematical Virtual Machine**”. It is designed as a technical lab notebook rather than a paper replacement.

Notebook design

- install cell first
- no hard-coded directories
- optional exports only
- shape-first visualizations
- no solver / SMT / SAT path
- explicit separation between **verified arithmetic**, **instrumented algorithmics**, and **manuscript-level interpretation**

What is covered

1. Mark 1 attractor arithmetic
2. BBP pointer engine and 32-bit word extraction
3. Instrumented SHA-256 round mechanics
4. Ground fold and NOP backbone
5. Sziklai coupling identity
6. Prime-root K-constant audit
7. Rotation / XOR / carry geometry
8. 33 Hz dual-phase model
9. Glass-key compression arithmetic

```
In [1]: # Install cell
        # Run this only if your environment does not already have the packages.
        %pip -q install mpmath numpy pandas plotly nbformat
```

WARNING: The directory '/home/oai/.cache/pip' or its parent directory is not owned or is not writable by the current user. The cache has been disabled. Check the permissions and owner of that directory. If executing pip with sudo, you should use sudo's -H flag.

Note: you may need to restart the kernel to use updated packages.

```
In [2]: from __future__ import annotations

import hashlib
import json
import math
import random
from pathlib import Path
from typing import List, Tuple, Dict

import mpmath as mp
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

# -----
# Configuration
# -----
EXPORT_ARTIFACTS = False
OUTPUT_DIR = Path("bbp_sha_vm_outputs")
RANDOM_SEED = 42
mp.mp.dps = 120
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

if EXPORT_ARTIFACTS:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 140)

MASK32 = 0xFFFFFFFF
MARK1_H = math.pi / 9
```

```
print(f"Configured. EXPORT_ARTIFACTS={EXPORT_ARTIFACTS}, output_dir={OUTPUT_DIR.resolve()}")
print(f"Mark 1 attractor H = pi/9 = {MARK1_H:.12f}")
```

Configured. EXPORT_ARTIFACTS=False, output_dir=/mnt/data/bbp_sha_vm_outputs
 Mark 1 attractor H = pi/9 = 0.349065850399

1. Mark 1 attractor arithmetic

The manuscript treats the Mark 1 attractor as the dimensionless target

$H = \frac{\pi}{9} \approx 0.3490658504$

This section computes:

- the exact floating value
- its distance from the engineering shorthand 0.35
- its relation to the mediant $7/20$
- a geometric visualization on the unit circle

```
In [3]: mark1_df = pd.DataFrame(
    {
        "quantity": ["pi/9", "0.35", "7/20", "difference(pi/9, 0.35)", "difference(pi/9, 7/20)"],
        "value": [
            MARK1_H,
            0.35,
            7/20,
            abs(MARK1_H - 0.35),
            abs(MARK1_H - 7/20),
        ],
    }
)
mark1_df
```

Out[3]:

	quantity	value
0	pi/9	0.349066
1	0.35	0.350000
2	7/20	0.350000
3	difference(pi/9, 0.35)	0.000934
4	difference(pi/9, 7/20)	0.000934

```
In [4]: theta = np.linspace(0, 2*np.pi, 600)
x = np.cos(theta)
y = np.sin(theta)

h_angle = MARK1_H
fig_mark1 = go.Figure()
fig_mark1.add_trace(go.Scatter(x=x, y=y, mode="lines", name="unit circle"))
fig_mark1.add_trace(
    go.Scatter(
        x=[0, math.cos(h_angle)],
        y=[0, math.sin(h_angle)],
        mode="lines+markers",
        name="H = pi/9",
    )
)
fig_mark1.add_trace(
    go.Scatter(
        x=[math.cos(0.35)],
        y=[math.sin(0.35)],
        mode="markers",
        name="0.35 shorthand",
        marker=dict(size=10),
    )
)
fig_mark1.update_layout(
    title="Mark 1 attractor geometry on the unit circle",
    xaxis_title="x",
    yaxis_title="y",
    yaxis_scaleanchor="x",
    width=700,
```

```

        height=600,
    )
    fig_mark1.show()

```

2. BBP pointer engine

We implement the Bailey–Borwein–Plouffe hex-digit extractor and then aggregate eight hex digits into 32-bit words.

For a starting hexadecimal offset d , the notebook computes the word

$$W_d = \sum_{k=0}^7 h_{d+k} \cdot 16^{7-k}$$

where h_n is the n -th hexadecimal digit of the fractional part of π .

```

In [5]: def bbp_hex_digit(d: int) -> int:
        """Return the d-th hexadecimal digit of the fractional part of pi (0-indexed)."""
        if d < 0:
            raise ValueError("d must be non-negative")
        s = mp.mpf("0")
        for j, coeff in [(1, 4), (4, -2), (5, -1), (6, -1)]:
            sj = mp.mpf("0")
            for k in range(d + 1):
                denom = 8 * k + j
                sj += mp.mpf(pow(16, d - k, denom)) / denom
            k = d + 1
            term = mp.power(16, d - k) / (8 * k + j)
            while abs(term) > mp.mpf("1e-80"):
                sj += term
                k += 1
                term = mp.power(16, d - k) / (8 * k + j)
            s += coeff * sj
        frac = s - mp.floor(s)
        return int(mp.floor(16 * frac))

def bbp_word32(d: int) -> int:
    word = 0
    for i in range(8):
        word = (word << 4) | bbp_hex_digit(d + i)
    return word

```

```
test_positions = [0, 8, 16, 24, 32, 64, 128]
bbp_words = [{"offset_hex_digit": d, "word_hex": f"0x{bbp_word32(d):08x}", "word_uint32": bbp_word32(d)} for d in test_positions]
bbp_df = pd.DataFrame(bbp_words)
bbp_df
```

```
Out[5]:
```

	offset_hex_digit	word_hex	word_uint32
0	0	0x243f6a88	608135816
1	8	0x85a308d3	2242054355
2	16	0x13198a2e	320440878
3	24	0x03707344	57701188
4	32	0xa4093822	2752067618
5	64	0x452821e6	1160258022
6	128	0x9216d5d9	2450970073

```
In [6]: expected_words = {
    0: 0x243F6A88,
    8: 0x85A308D3,
    16: 0x13198A2E,
    24: 0x03707344,
    32: 0xA4093822,
    64: 0x452821E6,
    128: 0x9216D5D9,
}

bbp_check = []
for d, expected in expected_words.items():
    actual = bbp_word32(d)
    bbp_check.append(
        {
            "offset": d,
            "expected_hex": f"0x{expected:08x}",
            "actual_hex": f"0x{actual:08x}",
            "match": actual == expected,
        }
    )
```

```
)
bbp_check_df = pd.DataFrame(bbp_check)
bbp_check_df
```

```
Out[6]:
```

	offset	expected_hex	actual_hex	match
0	0	0x243f6a88	0x243f6a88	True
1	8	0x85a308d3	0x85a308d3	True
2	16	0x13198a2e	0x13198a2e	True
3	24	0x03707344	0x03707344	True
4	32	0xa4093822	0xa4093822	True
5	64	0x452821e6	0x452821e6	True
6	128	0x9216d5d9	0x9216d5d9	True

```
In [7]: def word_to_bits(word: int, width: int = 32) -> list[int]:
        return [(word >> (width - 1 - i)) & 1 for i in range(width)]

bit_rows = []
for d in test_positions:
    w = bbp_word32(d)
    bit_rows.append(word_to_bits(w))
bit_matrix = np.array(bit_rows)

fig_bbp_bits = px.imshow(
    bit_matrix,
    aspect="auto",
    color_continuous_scale="Viridis",
    labels=dict(x="bit position", y="word index", color="bit"),
    title="Bit planes of BBP 32-bit words from the pi manifold",
)
fig_bbp_bits.update_yaxes(
    tickvals=list(range(len(test_positions))),
    ticktext=[f"d={d}" for d in test_positions],
)
fig_bbp_bits.show()
```

3. Instrumented SHA-256 round mechanics

We implement a pure-Python, single-block, fully instrumented SHA-256 compressor.

This gives direct access to:

- message schedule words
- round constants
- \$T1\$ and \$T2\$
- carry counts
- register trajectories
- final digest check against `hashlib`

```
In [8]: K = [
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90bffffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
]

H0 = [
    0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
    0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19,
]

def rotr(x: int, n: int) -> int:
    return ((x >> n) | ((x << (32 - n)) & MASK32)) & MASK32

def ch(x: int, y: int, z: int) -> int:
    return (x & y) ^ (~x & z) & MASK32

def maj(x: int, y: int, z: int) -> int:
    return (x & y) ^ (x & z) ^ (y & z)
```

```

def Sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def add32(*xs: int) -> Tuple[int, int]:
    total = sum(xs)
    return total & MASK32, total >> 32

def sha256_pad_one_block(msg: bytes) -> bytes:
    m1 = len(msg) * 8
    padded = msg + b"\x80"
    while (len(padded) % 64) != 56:
        padded += b"\x00"
    padded += m1.to_bytes(8, "big")
    if len(padded) != 64:
        raise ValueError("This helper expects a single 512-bit block after padding.")
    return padded

def schedule_from_block(block: bytes) -> List[int]:
    W = [int.from_bytes(block[i:i+4], "big") for i in range(0, 64, 4)]
    for t in range(16, 64):
        wt, _ = add32(sigma1(W[t-2]), W[t-7], sigma0(W[t-15]), W[t-16])
        W.append(wt)
    return W

def instrumented_sha256_one_block(msg: bytes) -> Dict[str, object]:
    block = sha256_pad_one_block(msg)
    W = schedule_from_block(block)
    a, b, c, d, e, f, g, h = H0
    rounds = []

    for t in range(64):
        T1, carry_T1 = add32(h, Sigma1(e), ch(e, f, g), K[t], W[t])
        T2, carry_T2 = add32(Sigma0(a), maj(a, b, c))

```

```

new_a, _ = add32(T1, T2)
new_e, _ = add32(d, T1)

rounds.append(
    {
        "t": t,
        "a": a, "b": b, "c": c, "d": d, "e": e, "f": f, "g": g, "h": h,
        "W": W[t], "K": K[t],
        "Sigma0(a)": Sigma0(a), "Sigma1(e)": Sigma1(e),
        "Maj": maj(a, b, c), "Ch": ch(e, f, g),
        "T1": T1, "T2": T2,
        "carry_T1": carry_T1,
        "carry_T2": carry_T2,
        "a_next": new_a,
        "e_next": new_e,
    }
)

a, b, c, d, e, f, g, h = new_a, a, b, c, new_e, e, f, g

final_state = []
for x0, x in zip(H0, [a, b, c, d, e, f, g, h]):
    s, _ = add32(x0, x)
    final_state.append(s)

digest = b"".join(x.to_bytes(4, "big") for x in final_state).hex()

return {
    "block_hex": block.hex(),
    "schedule": W,
    "rounds": rounds,
    "digest_hex": digest,
    "hashlib_hex": hashlib.sha256(msg).hexdigest(),
    "final_state": final_state,
}

```

```

trace_abc = instrumented_sha256_one_block(b"abc")
trace_abc["digest_hex"], trace_abc["hashlib_hex"], trace_abc["digest_hex"] == trace_abc["hashlib_hex"]

```

```

Out[8]: ('ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad',
         'ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad',
         True)

```

```
In [9]: rounds_abc_df = pd.DataFrame(trace_abc["rounds"])
rounds_abc_df.head(8)[["t", "a", "e", "W", "K", "T1", "T2", "carry_T1", "carry_T2", "a_next", "e_next"]]
```

	t	a	e	W	K	T1	T2	carry_T1	carry_T2	a_next	e_next
0	0	1779033703	1359893119	1633837952	1116352408	1423593704	143694565	1	1	1567288269	4197074466
1	1	1567288269	4197074466	0	1899447441	1012893207	504058774	1	1	1516951981	2026797449
2	2	1516951981	2026797449	0	3049323471	1036094310	2332146753	2	0	3368241063	4180228587
3	3	3368241063	4180228587	0	3921009573	3134595561	444257405	1	1	3578852966	618661968
4	4	3578852966	618661968	0	961987163	3863131768	503178226	1	1	71342698	1135452741
5	5	71342698	1135452741	0	1508970993	383223552	342524661	2	1	725748213	1900175533
6	6	725748213	1900175533	0	2453635748	3529792602	312387366	1	0	3842179968	2603066369
7	7	3842179968	2603066369	0	2870763221	924091411	1317795660	1	0	2241887071	207977081

4. Ground fold and NOP backbone

The manuscript repeatedly treats the first $T2$ as a universal ground fold:

$$T2_0 = \text{Sigma}_0(H_0[a]) + \text{Maj}(H_0[a], H_0[b], H_0[c])$$

We compute that directly and then run the message-free NOP backbone by setting all $W_t = 0$.

```
In [10]: T2_0, carry_T2_0 = add32(Sigma0(H0[0]), maj(H0[0], H0[1], H0[2]))
print(f"T2_0 = 0x{T2_0:08x}, carry={carry_T2_0}")
```

```
T2_0 = 0x08909ae5, carry=1
```

```
In [11]: def nop_backbone() -> pd.DataFrame:
    a, b, c, d, e, f, g, h = H0
    rows = []
    for t in range(64):
        T1, carry_T1 = add32(h, Sigma1(e), ch(e, f, g), K[t], 0)
        T2, carry_T2 = add32(Sigma0(a), maj(a, b, c))
```

```

a_next, _ = add32(T1, T2)
e_next, _ = add32(d, T1)
rows.append(
    {
        "t": t,
        "a": a, "e": e, "T1": T1, "T2": T2,
        "carry_T1": carry_T1,
        "carry_T2": carry_T2,
        "a_next": a_next,
        "e_next": e_next,
    }
)
a, b, c, d, e, f, g, h = a_next, a, b, c, e_next, e, f, g
return pd.DataFrame(rows)

nop_df = nop_backbone()
nop_df.head(10)

```

```

Out[11]:

```

	t	a	e	T1	T2	carry_T1	carry_T2	a_next	e_next
0	0	1779033703	1359893119	4084723048	143694565	0	1	4228417613	2563236514
1	1	4228417613	2563236514	1635958436	425108460	1	1	2061066896	2649862678
2	2	2061066896	2649862678	169065078	3922309577	3	0	4091374655	3313199355
3	3	4091374655	3313199355	647171938	3817972296	2	1	170176938	2426205641
4	4	170176938	2426205641	815959133	402470433	2	1	1218429566	749409450
5	5	1218429566	749409450	574139654	1232743809	2	1	1806883463	2635206550
6	6	1806883463	2635206550	2246705563	276089159	1	1	2522794722	2043112922
7	7	2522794722	2043112922	2315401910	3561444605	1	0	1581879219	2485578848
8	8	1581879219	2485578848	989683262	3935381372	1	0	630097338	2208112828
9	9	630097338	2208112828	3538563424	3035821374	1	0	2279417502	1050479591

```

In [12]: carry_sig_bits = "".join(str(int(x)) for x in nop_df["carry_T2"].tolist())
carry_sig_hex = hex(int(carry_sig_bits, 2))
print("NOP T2 carry signature (binary):")

```

```

print(carry_sig_bits)
print("\nNOP T2 carry signature (hex):")
print(carry_sig_hex)
print("\nHamming weight:", nop_df["carry_T2"].sum(), "/ 64")

```

NOP T2 carry signature (binary):
 1101111000011010010101000101011010001101011000001111011110110110

NOP T2 carry signature (hex):
 0xde1a54568d60f7b6

Hamming weight: 34 / 64

```

In [13]: fig_carry = px.bar(
    nop_df,
    x="t",
    y="carry_T2",
    title="NOP backbone T2 carry channel",
)
fig_carry.update_layout(height=400)
fig_carry.show()

```

5. Sziklai coupling identity

We verify the algebraic identity

$$a_{t+1} - e_{t+1} \equiv T2_t - d_t \pmod{2^{32}}$$

for every round, first on the `abc` trace and then across a sample of random one-block messages.

```

In [14]: def mod32_sub(x: int, y: int) -> int:
    return (x - y) & MASK32

coupling_checks = []
for row in trace_abc["rounds"]:
    lhs = mod32_sub(row["a_next"], row["e_next"])
    rhs = mod32_sub(row["T2"], row["d"])
    coupling_checks.append(lhs == rhs)

```

```
print("ABC trace: violations =", sum(not x for x in coupling_checks))
```

ABC trace: violations = 0

```
In [15]: def random_one_block_message(max_len: int = 55) -> bytes:
          n = random.randint(0, max_len)
          return bytes(random.getrandbits(8) for _ in range(n))

violations = 0
tested_rounds = 0
num_messages = 200

for _ in range(num_messages):
    msg = random_one_block_message()
    trace = instrumented_sha256_one_block(msg)
    for row in trace["rounds"]:
        lhs = mod32_sub(row["a_next"], row["e_next"])
        rhs = mod32_sub(row["T2"], row["d"])
        if lhs != rhs:
            violations += 1
            tested_rounds += 1

pd.DataFrame(
    [{"messages": num_messages, "tested_rounds": tested_rounds, "violations": violations}]
)
```

```
Out[15]:
```

	messages	tested_rounds	violations
0	200	12800	0

6. Prime-root rail audit

This section checks that the SHA-256 round constants K_t match the fractional cube roots of the first 64 primes, scaled into 32-bit words.

```
In [16]: def primes_first_n(n: int) -> List[int]:
          out = []
          candidate = 2
```

```

while len(out) < n:
    is_prime = True
    for p in out:
        if p * p > candidate:
            break
        if candidate % p == 0:
            is_prime = False
            break
    if is_prime:
        out.append(candidate)
        candidate += 1 if candidate == 2 else 2
return out

def expected_k_from_prime(p: int) -> int:
    x = mp.root(mp.mpf(p), 3)
    frac = x - mp.floor(x)
    return int(mp.floor(frac * (2 ** 32)))

primes64 = primes_first_n(64)
audit_rows = []
for i, p in enumerate(primes64):
    expected = expected_k_from_prime(p)
    actual = K[i]
    audit_rows.append(
        {
            "index": i,
            "prime": p,
            "expected_hex": f"0x{expected:08x}",
            "actual_hex": f"0x{actual:08x}",
            "delta": actual - expected,
            "match": actual == expected,
        }
    )
)
k_audit_df = pd.DataFrame(audit_rows)
k_audit_df.head(10)

```

```
Out[16]:
```

	index	prime	expected_hex	actual_hex	delta	match
0	0	2	0x428a2f98	0x428a2f98	0	True
1	1	3	0x71374491	0x71374491	0	True
2	2	5	0xb5c0fbcf	0xb5c0fbcf	0	True
3	3	7	0xe9b5dba5	0xe9b5dba5	0	True
4	4	11	0x3956c25b	0x3956c25b	0	True
5	5	13	0x59f111f1	0x59f111f1	0	True
6	6	17	0x923f82a4	0x923f82a4	0	True
7	7	19	0xab1c5ed5	0xab1c5ed5	0	True
8	8	23	0xd807aa98	0xd807aa98	0	True
9	9	29	0x12835b01	0x12835b01	0	True

```
In [17]: k_audit_df["match"].all(), k_audit_df["delta"].abs().sum()
```

```
Out[17]: (np.True_, np.int64(0))
```

```
In [18]: fig_k = px.scatter(
    k_audit_df,
    x="index",
    y="prime",
    hover_data=["expected_hex", "actual_hex", "match"],
    title="Prime rails used to generate the SHA-256 K constants",
)
fig_k.show()
```

7. Shape-first views of ROTR, XOR, and ADD

The manuscript frames:

- **ROTR** as spatial relabeling

- **XOR** as interference / cancellation
- **ADD** as a carry wave

This section builds small, direct visualizations for each.

```
In [19]: def rotation_matrix(width: int, shift: int) -> np.ndarray:
    M = np.zeros((width, width), dtype=int)
    for src in range(width):
        dst = (src + shift) % width
        M[dst, src] = 1
    return M

rot_M = rotation_matrix(32, 7)
fig_rot = px.imshow(
    rot_M,
    aspect="auto",
    color_continuous_scale="Blues",
    labels=dict(x="source bit", y="destination bit", color="route"),
    title="ROTR geometry as a permutation matrix (32-bit, shift=7)",
)
fig_rot.show()
```

```
In [20]: x = 0xDEADBEEF
y = 0xBEEFDEAD
xb = np.array(word_to_bits(x))
yb = np.array(word_to_bits(y))
xorb = xb ^ yb

xor_vis = pd.DataFrame(
    {
        "bit": np.arange(32),
        "x": xb,
        "y": yb,
        "xor": xorb,
    }
)

fig_xor = go.Figure()
fig_xor.add_trace(go.Bar(x=xor_vis["bit"], y=xor_vis["x"], name="x bits"))
fig_xor.add_trace(go.Bar(x=xor_vis["bit"], y=xor_vis["y"], name="y bits"))
```

```

fig_xor.add_trace(go.Scatter(x=xor_vis["bit"], y=xor_vis["xor"], mode="lines+markers", name="x XOR y"))
fig_xor.update_layout(
    barmode="group",
    title="XOR viewed as bit-pattern interference",
    xaxis_title="bit position",
    yaxis_title="bit value",
    height=450,
)
fig_xor.show()

```

```

In [21]: def carry_path_length(a: int, b: int) -> int:
    carry = 0
    longest = 0
    current = 0
    for i in range(32):
        ai = (a >> i) & 1
        bi = (b >> i) & 1
        new_carry = (ai & bi) | (carry & (ai ^ bi))
        if new_carry:
            current += 1
            longest = max(longest, current)
        else:
            current = 0
        carry = new_carry
    return longest

carry_samples = []
for _ in range(400):
    a = random.getrandbits(32)
    b = random.getrandbits(32)
    carry_samples.append(carry_path_length(a, b))

carry_len_df = pd.DataFrame({"carry_path_length": carry_samples})
fig_add = px.histogram(
    carry_len_df,
    x="carry_path_length",
    nbins=32,
    title="Carry-path length distribution for random 32-bit additions",
)
fig_add.show()

```

8. 33 Hz dual-phase model

This section visualizes the manuscript's dual-phase 33 Hz rhythm as a simple two-state stroke:

- alive phase: 16.5 Hz effective half-cycle
- dead phase: 16.5 Hz effective half-cycle

```
In [22]: fs = 5000
t = np.linspace(0, 0.3, int(0.3 * fs), endpoint=False)
freq = 33.0
carrier = np.sin(2 * np.pi * freq * t)
alive = (carrier >= 0).astype(float)
dead = 1.0 - alive

phase_df = pd.DataFrame({"t": t, "carrier": carrier, "alive": alive, "dead": dead})

fig_phase = go.Figure()
fig_phase.add_trace(go.Scatter(x=phase_df["t"], y=phase_df["carrier"], mode="lines", name="33 Hz carrier"))
fig_phase.add_trace(go.Scatter(x=phase_df["t"], y=phase_df["alive"], mode="lines", name="alive phase"))
fig_phase.add_trace(go.Scatter(x=phase_df["t"], y=phase_df["dead"], mode="lines", name="dead phase"))
fig_phase.update_layout(
    title="Dual-phase 33 Hz render model",
    xaxis_title="time (s)",
    yaxis_title="state",
    height=450,
)
fig_phase.show()
```

9. 112-byte glass-key compression arithmetic

This section only evaluates the arithmetic of the stated footprint.

It does **not** independently validate the compression claim itself.

```
In [23]: one_gib = 1024 ** 3
glass_key_bytes = 112
ratio = one_gib / glass_key_bytes
```

```
compression_df = pd.DataFrame(
    {
        "source_bytes": [one_gib],
        "compressed_bytes": [glass_key_bytes],
        "compression_ratio": [ratio],
        "ratio_million_to_one": [ratio / 1_000_000],
    }
)
compression_df
```

```
Out[23]:
```

	source_bytes	compressed_bytes	compression_ratio	ratio_million_to_one
0	1073741824	112	9.586981e+06	9.586981

```
In [24]: fig_comp = go.Figure()
fig_comp.add_trace(go.Bar(x=["1 GiB source", "112-byte footprint"], y=[one_gib, glass_key_bytes], name="bytes"))
fig_comp.update_layout(
    title="Compression arithmetic only: source vs. stated footprint",
    yaxis_title="bytes",
    height=450,
)
fig_comp.show()
```

10. Curated optional exports

This notebook does not dump files by default.

If `EXPORT_ARTIFACTS = True`, the cell below writes a small, curated set of artifacts:

- BBP word table
- SHA `abc` round trace
- NOP backbone
- K-constant audit
- notebook summary JSON

```
In [25]: if EXPORT_ARTIFACTS:
bbp_df.to_csv(OUTPUT_DIR / "bbp_word_table.csv", index=False)
```

```

rounds_abc_df.to_csv(OUTPUT_DIR / "sha_abc_round_trace.csv", index=False)
nop_df.to_csv(OUTPUT_DIR / "nop_backbone.csv", index=False)
k_audit_df.to_csv(OUTPUT_DIR / "k_constant_audit.csv", index=False)

summary = {
    "mark1_h": MARK1_H,
    "T2_0_hex": f"0x{T2_0:08x}",
    "abc_digest_hex": trace_abc["digest_hex"],
    "hashlib_match": trace_abc["digest_hex"] == trace_abc["hashlib_hex"],
    "nop_t2_carry_signature_hex": carry_sig_hex,
    "k_audit_all_match": bool(k_audit_df["match"].all()),
    "glass_key_ratio": ratio,
}
with open(OUTPUT_DIR / "summary.json", "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2)

print(f"Wrote curated artifacts to: {OUTPUT_DIR.resolve()}")
else:
    print("EXPORT_ARTIFACTS is False. No files were written.")

```

EXPORT_ARTIFACTS is False. No files were written.

11. Summary

This notebook gives a full executable treatment of the manuscript's main computational structures without embedding the paper itself:

- BBP as a direct 32-bit word extractor
- SHA-256 as an instrumented 64-round fold engine
- universal ground fold $T2_0 = 0x08909ae5$
- NOP backbone carry signature
- Sziklai coupling identity across sampled executions
- exact prime-root audit of the K rails
- shape-first views of rotation, interference, and carry propagation
- arithmetic treatment of the 33 Hz dual-phase model and the 112-byte footprint

Use it as a working lab notebook, not as a static appendix.